Real-world (one reusable session) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `session_reuse_practice.py`: headers passed by hand to every call are a typo waiting to happen — one call in five forgets the `Authorization` header, and nothing tells you until a request mysteriously gets rejected. One shared `Session`, configured once, makes that mistake structurally impossible. **If not:** `http_client.py` in the Build Task would open a new connection and re-specify headers on every single call, slower and more fragile than it needs to be.

Solution

```
# session_reuse_practice.py
import requests


# The one thing this exercise teaches: a reusable session with the auth
# header set once, so no later call can forget it.
def make_authenticated_session(token):
    session = requests.Session()
    session.headers.update({"Authorization": f"Bearer {token}"})
    return session


# Build the session once, then every call carries the header automatically.
session = make_authenticated_session("fake-token")

urls = [
    "https://api.github.com",
    "https://api.github.com/zen",
    "https://api.github.com/octocat",
]
for url in urls:
    # session.get(...), not requests.get(...), is what sends the header
    response = session.get(url)
    print(f"{url} -> {response.status_code}")

# response.request.headers reads back what actually went over the wire
sent_header = response.request.headers.get("Authorization")
print(f"header actually sent on the last request: {sent_header}")
```

Expected output:

```
https://api.github.com -> 200
https://api.github.com/zen -> 200
https://api.github.com/octocat -> 200
header actually sent on the last request: Bearer fake-token
```

`response.request.headers` reads back the *actual* request object `requests` sent, not the session's own dict — confirming the header genuinely went out over the wire, not just that it's set somewhere in Python.